



Tecnológico de Monterrey

Instituto Tecnológico y de Estudios Superiores de Monterrey

Campus Santa Fe

Modelación de sistemas multiagentes con gráficas computacionales (Gpo 302)

Nombres de los profesores:

Octavio Navarro Hinojosa

Gilberto Echeverría Furió

Actividad:

Movilidad urbana

Alumno:

Rebeca Davila Araiza | A01029805

José Ángel De La Cruz Alonso | A01772695

Fecha:

4 de diciembre del 2025

Problema que se está resolviendo, y la propuesta de solución.

La movilidad urbana en México se ha convertido en un todo un desafío debido al constante uso de los automóviles. En gran parte son responsables de graves consecuencias económicas, sociales y ambientales. En las últimas décadas, los Kilómetros-Auto Recorridos aumentaron de 106 millones en 1990 a 339 millones en 2010, lo cual refleja el incremento en contaminación atmosférica, congestión vial, accidentes y enfermedades relacionadas. Este método de transporte limita la calidad de vida de millones de personas y obstaculiza el desarrollo sostenible de las ciudades. Ante este panorama, es importante replantear cómo se gestiona y se entiende la movilidad urbana en el país.

Es por eso que este reto propone abordar esta problemática mediante una propuesta basada en una simulación gráfica del tráfico con un sistema multiagente, con el propósito de analizar dinámicas de congestión y generar propuestas que contribuyan a mejorar la movilidad en las ciudades mexicanas. Es por eso que se integró el algoritmo de A* con el fin de que los agentes busquen una ruta óptima sin colisionar o provocar un embotellamiento, para lograr esto usamos la siguiente fórmula $f(n)=g(n)+h(n)$.

Objetivo General del Agente (Carro)

El objetivo general del agente carro es navegar de manera autónoma desde su posición inicial hasta un destino aleatorio dentro de la ciudad, siguiendo las reglas de tránsito y adaptándose a las condiciones del entorno, el carro deberá de calcular la ruta más eficiente utilizando el algoritmo de A* considerando la distancia, congestión vehicular, estado de los semáforos y la presencia de edificios (obstáculos), durante su trayectoria el agente debe de respetar las direcciones de las carreteras, detenerse en los semáforos y evitar colisiones.

Además el carro implementa comportamientos inteligentes como el cambio de carril cuando detecta congestión, permitiéndole utilizar carriles alternativos para mantener un flujo, también recalcula su ruta periódicamente para adaptarse a cambios en el tráfico, optimizando su tiempo de llegada, cuando alcanza su destino el agente desaparece del sistema registrando su llegada exitosa.

Características de agentes

Diseño de los agentes:

Resumen

Agente	Objetivo	Capacidad Efectora	Percepción	Proactividad	Métricas de Desempeño
Carro	Llegar a su destino evitando obstáculos, semáforos, congestión y colisiones.	Moverse, cambiar carril, recalcular la ruta de A*	Detección de semáforos, carros cercanos y obstáculos cercanos	Navegación hacia el destino, respeto de semáforos, evita colisiones, recalcula la ruta periódicamente.	- Llegada exitosa - Cantidad de recalculos
Semáforo	Controlar flujo vehicular	Cambiar de color/estado cada cierto tiempo	No percibe nada solo cambia su estado cada cierto tiempo	Control del semáforo	N/A
Destino	Indicar el punto final del carro	Seguir el objetivo de los carros	N/A	N/A	N/A
Obstáculo	Bloquear el paso	Ser un objeto fijo	N/A	N/A	N/A
Carretera	Definir donde pueden moverse los carros y en qué dirección	Indicar la dirección permitida del movimiento	N/A	N/A	N/A

Detallado

Capacidades Efectoras

- Los carros pueden moverse de su celda actual a una celda cercana dentro del grid, siguiendo las direcciones permitidas por la carretera, cuando se mueve actualiza "self.cell" a la nueva posición.
- Si hay un carro bloqueando el camino directo, el otro carro puede moverse en diagonal hacia carriles alternativos.
- El agente debe de quedarse en su celda actual sin moverse al detectar el semaforo en color rojo o amarillo.
- Cuando el carro alcanza su celda desaparece completamente del grid.

Percepción

Cada carro en la simulación puede percibir si:

- Hay un carro enfrente de este y en las esquinas delanteras
- Ver si hay un semáforo en rojo o amarillo para detenerse
- Hay obstáculos en las celdas vecinas

Proactividad

Los carros actúan con base a las siguientes metas:

- Alcanzar su destino asignado
- Seguir la ruta de A*
- Evitar quedarse atascado en el tráfico
- Respetar las reglas de tráfico
- Evitar destinos ajenos

Reactividad

El carro reacciona a los siguientes elementos del entorno:

- Evita obstáculos si los encuentra como vecinos.
- Se detiene cuando los semaforos estan en rojo o amarillo
- Se detiene cuando hay otro carro bloqueando su camino
- Se muere cuando llega a su destino
- Ajusta su dirección según el movimiento realizado

Métricas de Desempeño

Solamente para un agente, ya que implementar estas métricas a más de uno vuelve la simulación más lenta:

- Total de carros que llegaron a su destino
- Cantidad de recálculos realizados
- Distancia total recorrida

Arquitectura de Subsunción

- Capa 3 – Navegación hacia el destino

Dentro del nivel 3 lo que hará es que cuando el agente no tiene restricciones de las capas inferiores su prioridad es seguir la ruta calculada por A* para llegar a su destino.

```
def inicializar_destino(self): #asigna destino aleatorio y calcula A*
    if not self.camino_calculado:
        self.destino = self.elegir_objetivo()
    if self.destino:
        self.camino = AStar.buscar_camino(self.cell, [self.destino], self.model, self.destino)
        self.camino_calculado = True # marca que ya se calculo
```

Básicamente empieza verificando si el agente ya tienen un camino calculado, en donde para eso checa “camino_calculado”, si no lo tiene entonces llama a “inicializar_destino()” que busca todos los destinos disponibles y eligiendo uno aleatorio luego calcula la ruta más óptima usando A* hacia ese destino, con cada step el agente revisa si ya llego hacia su destino comparando su celda actual con “self.destino”, si es así incrementa el contador de carros exitosos y se elimina el modelo.

- Capa 2 – Evasión de congestión

Dentro del nivel 2 cuando el agente detecta otro vehículo bloqueado la siguiente celda cambia su prioridad y busca un carril alternativo para evitar quedarse atascado.

```
# verificar que no haya otro carro en la siguiente celda
hay_carro = any(isinstance(agent, Car) for agent in next_cell.agents)

# si hay carro bloqueando intentar cambiar de carril
if hay_carro:
    carriles = self.obtener_carriles_alternativos()
    # ordenar carriles por cantidad de carros en donde priorizamos en donde haya menos carros
    carriles.sort(key=lambda c: sum(1 for a in c.agents if isinstance(a, Car)))

    for carril in carriles:
        # verificar que el carril este libre
        tiene_carro = any(isinstance(agent, Car) for agent in carril.agents)
        if not tiene_carro:
            # actualizar direccion del carro para rotacion 3D antes de cambiar de carril
            self.actualizar_direccion(carril)
            # cambiar de carril
            self.cell = carril
            self.pasos_sin_mover = 0
            return

    # si no hay carril libre, quedarse quieto
    self.pasos_sin_mover += 1
    return
```

Aquí básicamente revisa si hay algún agente carro en la siguiente celda del camino, si lo es entonces el agente llama a “obtener_carriles_alternativos()” que identifica las posiciones diagonales disponibles según la dirección de la carretera, el agente se mueve hacia estos carriles y verifica que estén libres.

- Capa 1 – Respetar semáforos

Dentro del nivel cuando el agente detecta un semaforo en rojo o amarillo en la siguiente celda, el carro se mantendra quieto hasta que el semaro cambie a verde.

```

# verificar si hay un semaforo en rojo o amarillo en la siguiente celda
hay_semaforo_rojo = False
for agent in next_cell.agents:
    if isinstance(agent, Traffic_Light):
        if agent.state == 0 or agent.state == 2: # rojo o amarillo
            hay_semaforo_rojo = True
            break

# si hay semaforo rojo/amarillo en la siguiente celda no puede entrar
if hay_semaforo_rojo:
    self.pasos_sin_mover += 1
    return

```

Basicamente revisa si hay un agente de tipo “traffic_light” en la siguiente celda del camino y verifica su estado si esta en rojo que es el 0 y amarillo si esta en 2 y solo se mueve cuando esta el semaforo esta en verde.

- Capa 0 – Evitar colisiones

Dentro del nivel 0 el agente evita colisiones con los edificios (obstáculos) antes de realizar cualquier movimiento.

```

for vecino in vecinos_permitidos:
    # verifica obstaculos
    if any(isinstance(agent, Obstacle) for agent in vecino.agents):
        continue

```

Para evitar colisiones se hace de dos maneras con el fin de garantizar que el agente por nada del mundo intente moverse a una celda bloqueada primero calcula la ruta con A* descartando los obstáculos durante la planificación y segundo al buscar carriles alternativos de manera diagonal viendo que no haya obstáculos en tiempo real.

Características del Ambiente

- Es accesible ya que el carro detecta el estado de su celda y las celdas vecinas, detectando la presencia de otros carros, semáforos, obstáculos y destinos, lo que da información suficiente para tomar decisiones de movimiento y cambio de carril.
- Es no determinista ya que las acciones no siempre tienen resultados predecibles, por ejemplo moverse a otra celda puede ser exitoso o no dependiendo si otro carro llega primero a esa posición.
- Es secuencial ya que las decisiones actuales afectan estados futuros

- Es dinámico porque el ambiente cambia constantemente debido a que los semáforos cambian de estado cada cierto tiempo.
- Es discreto ya que se compone de celdas individuales con estados bien definidos.

Dimensiones

El ambiente se modela en un grid en donde las dimensiones se calculan a partir del archivo del mapa llamado “2025_base.txt” en donde el ancho corresponde a la longitud de la primera línea y la altura al número total de líneas del archivo.

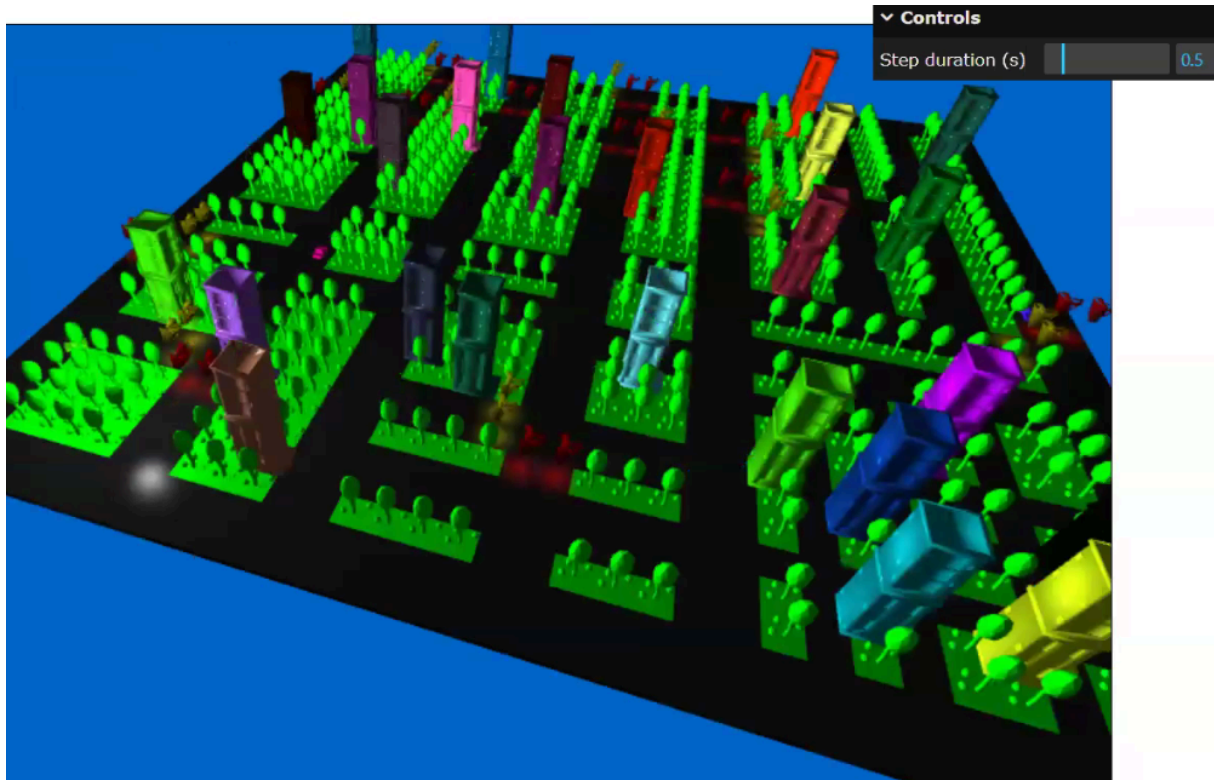
```
self.width = len(lines[0])
self.height = len(lines)
```

Elementos del ambiente

Elemento	Representación	Ubicación	Funcion
Edificios	Obstacle	Distribuidos por todo el mapa según el carácter #	Obstáculos para los coches
Carreteras	Road	Distribuidos por todo el mapa según los caracteres v, ^, >, <	Definir las direcciones del tránsito permitidas
Semáforos	Traffic_light	Distribuidos por todo el mapa según el carácter S	Regular el flujo de tráfico
Destinos	Destination	Distribuidos por todo el mapa según el carácter D	Objetivos de los carros
Carros	Car	Inicio en las esquinas	Llegar hacia los destinos

Ambiente en 3D

La simulación en 3D muestra el comportamiento de los agentes en un entorno urbano con edificios color café/gris, carreteras que es la parte oscura con árboles, semáforos que son los que cambian de color entre el rojo, amarillo y verde, dentro de esta simulación se observa como los agentes navegan simultáneamente hacia sus destinos respetando las direcciones de las carreteras y evitando obstáculos, los carros cuentan con diferentes colores entre los que se encuentran el amarillo, azul, morado y rojo.



Conclusiones

Durante la realización de este proyecto nos encontramos con varios retos al darle inteligencia a los agentes y reactividad, lo cual resultó en bastantes ajustes para lograr que todo el comportamiento funcionara y se pudiera integrar correctamente a la modelación.

A Pesar de los avances alcanzados todavía tenemos áreas de oportunidad, durante la simulación que se realizó de “Step 1” pudimos ver que a pesar que los carros si avanzaban se quedaban detenidos por mucho tiempo, considero que la causa de esto pudo haber sido el semáforo en donde se configuro el amarillo como el color rojo creo que si se hubiera configurado para que durara un poco menos el semaforo amarillo en vez del mismo tiempo que el rojo tal vez la simulacion de “Step 1” hubiera salido un poco mas natural, algo que en su momento se queria mejorar el contador de “pasos_sin_mover” en donde la idea era que con ese contador

empezara a tomar decisiones pero despues como que la idea no me resulto factible porque si en embotellamiento te quedas ahi por 10 o 15 steps pues no tiene mucho sentido volver a calcular si estas ahi parado sin hacer nada pero creo que este contador podria ser util en futuras mejoras.